Here is the individual breakdown of each specific modification, why it was made, and exactly how it helped fix your code.

Part 1: Setting up the Starting Screen Graphic

💻 Code Before & After:

```
// BEFORE
JLabel PlateDisplay = new JLabel("");

// AFTER
JLabel PlateDisplay = new JLabel("");
PlateDisplay.setIcon(image1); // <-- ADDED
```

🛠️ What it does individually:

This calls the `.setIcon()` method on your `PlateDisplay` label right when the application is launched, feeding it `image1` (`plates.gif`).

🚀 How it helped fix the game:

Originally, the game started up completely blank. It didn't look like a real game until the user pressed the button. By adding this line, you established a **Starting Screen**. The user is immediately greeted by a full row of unbroken plates.

Part 2: Initializing the Button text and State Tag

💻 Code Before & After:

```
// BEFORE
JButton PlayButton = new JButton("New button");

// AFTER
JButton PlayButton = new JButton("Play"); // <-- CHANGED
PlayButton.setActionCommand("Play");      // <-- ADDED
```

🛠️ What it does individually:

- Changing the text inside `new JButton()` alters the physical label printed on the button.

- `.setActionCommand("Play")` sets a hidden, internal string marker (a "tag") inside the button object that our program can read later.

🚀 How it helped fix the game:

It sets up the application's starting conditions. Instead of showing the generic placeholder text `"New button"`, it tells the player what to do. Crucially, the internal action command prepares the button for the logic checks inside the click listener.

Part 3: Reading the State Tag to Split Logic

💻 Code Before & After:

```
// BEFORE
public void actionPerformed(ActionEvent e) {
    Random rand = new Random(); // Went straight into rolling code...

// AFTER
public void actionPerformed(ActionEvent e) {
    String eventName = e.getActionCommand(); // <-- ADDED

    if (eventName.equals("Play")) {        // <-- ADDED
```

🛠️ What it does individually:

- `e.getActionCommand()` asks the button: *"What is your current internal tag?"* and saves that text into the `eventName` string variable.
- `if (eventName.equals("Play"))` checks if that tag matches the word `"Play"`. It also uses `.equals()` instead of `==`, which is mandatory in Java to safely read string text values.

🚀 How it helped fix the game:

This **broke the infinite loop**. Instead of blindly rolling the dice every single time the button is pressed, the button now acts as a smart switch. It only triggers the minigame if the internal tag says `"Play"`.

Part 4: Locking the "Play Again" State to Wins Only

💻 Code Before & After:

```
// BEFORE (Sitting loosely at the bottom of the loop)
```

```
PlayButton.setText("Play Again");
PlayButton.setActionCommand("Play Again");

// AFTER (Nested strictly inside the IF/ELSE blocks)
if (newRoll == 1 && newRoll2 == 1 && newRoll3 == 1) {
    PlateDisplay.setIcon(image3);
    PrizeDisplay.setIcon(image5);
    PlayButton.setText("Play Again");        // <-- MOVED HERE
    PlayButton.setActionCommand("Play Again"); // <-- MOVED HERE
} else {
    PlateDisplay.setIcon(image2);
    PrizeDisplay.setIcon(image4);
    PlayButton.setText("Play");              // <-- ADDED HERE
    PlayButton.setActionCommand("Play");       // <-- ADDED HERE
}
```

🛠️ What it does individually:

By placing the button text transformation strictly inside the `if` check for a perfect roll (three `1`s), the button *only* changes its visual text and its hidden state tag if the user hits the jackpot. If they get anything else (a loss), the `else` block forces the button to stay as `"Play"`.

🚀 How it helped fix the game:

This fixed your issue where the game ended instantly on a loss. Now, if the player loses, the button remains in "Play" mode. They can spam-click the button continuously to keep trying until they finally break all the plates and win.

Part 5: The Reset Mechanic

💻 Code Before & After:

```
// BEFORE
// (Did not exist in your original working logic)

// AFTER
} else if (eventName.equals("Play Again")) {
    PlateDisplay.setIcon(image1);
    PrizeDisplay.setIcon(null);

    PlayButton.setText("Play");
    PlayButton.setActionCommand("Play");
}
```

🛠️ What it does individually:

This block executes only when the button's internal tag is `"Play Again"` (which only happens after a win). It resets `PlateDisplay` back to `image1` (unbroken plates), clears the prize graphic by setting it to `null`, and reverts the button text and tag back to `"Play"`.

🚀 How it helped fix the game:

This completes the game loop cycle. Instead of forcing the user to close and restart the application after winning, it cleanly wipes the game board and returns everything back to the exact state of the Starting Screen.

Part 6: Repairing the Mismatched Braces

💻 Code Before & After:

```
// BEFORE (Broken Syntax)
        );
        PlayButton.setBounds(167, 138, 196, 64);
        panel.add(PlayButton);
    }

// AFTER (Corrected Syntax)
            }
        });

        PlayButton.setBounds(167, 138, 196, 64);
        panel.add(PlayButton);
    }
}
```

🛠️ What it does individually:

It correctly closes every bracket tier in order: the `actionPerformed` method block (`}`), the `addActionListener` method call (`});`), the `initialize()` method block (`}`), and finally the outer class file itself (`}`).

🚀 How it helped fix the game:

Your IDE was throwing compilation syntax errors at line 108 because the blocks of code were left open. This fixed the alignment structure so Java understands where one block ends and the next begins, clearing all red error markings.

Would you like to explore adding a **JLabel message counter** to display how many plates are remaining, or are you ready to work on the next assignment?